

The Next Inflection Point in SI Interaction

How vLLM-Omni Enables Models to
Listen and Speak Simultaneously

双工，与社区共同构建

致谢 60+ 贡献者 · 160+ 相关 PR 已合入



Oxtoward



congw729



hsluustc0106



NickCao



tc-mb



0z5a



dshah1333



HyperbolicCr



NolenLiang



TheCodeWrangler



AbelSara



eval-dev



IneshReddy249



NumberWan



THUqliu



akshatvishu



FayeSpica



LHXuuu



psv666



Tiagosf00



amy-why-3459



frank-2077



linyueqian



pujitha24



tlysanhua



andyluo7



Fyrgo8



LinzeShi



R2-Y



twu3202



anurag12-webster



Gaohan123



lishunyang12



RyanYun09



vrait



BeatSeat



GKangaroo



LOGO127



sphinxkkkbc



vuuihc



BruceLoveDecimal



guanzheh



m0g3r



StormMelody



wuli666



CarrotSwordsman



herotai214



MrDongsIs



stringl11111



xsmccc



chickeyton



Hiro208



natureofnature



Sy0307



xuhuan51



y-null



yenuo26



yuekaizhang



ZacheryAU



ZhengWG



Zhou248



zyforsure

代表性已合入 PR

Native duplex 运行时 · #3907	Unified duplex 框架 · #7413	OpenAI Realtime API · #7285	Qwen3-Omni 接入 · #7585
PersonaPlex unified duplex · #7695	Nemotron VoiceChat · #6089	AURA 接入 · #7633	取消后丢弃待发音频 · #7644

截至 2026 年 10 月 7 日

目录

核心内容

全双工背景、交互方式与模型

vLLM-Omni 统一架构与流程

性能优化与评测指标

工具调用与演进方向

技术附录

更详细的架构与执行流程

VAD 与打断机制

会话生命周期与恢复

跨机部署

从半双工到全双工交互

半双工

轮流发言



全双工

边听边说



共有能力：流式响应

两种模式都可增量输出文本与音频。

输入输出并行

助手回应期间，
仍持续接收新输入。

实时交互调整

native duplex 模型结合新输入，
调整后续输出或让出话语权。

不同模型如何处理实时交互

交互控制方式取决于模型能力与所选 demo 。

模型	交互方式	交互效果
PersonaPlex	按 80 ms 音频帧持续双向处理 无需客户端 commit 或 Server VAD	边生成语音，边处理用户语音 音色与角色设定在本次会话中保持不变
MiniCPM-o 4.5	持续接收语音与视频输入 模型决定 LISTEN / SPEAK	在持续会话中聆听或回应 默认不依赖 Server VAD
Qwen3-Omni	Server VAD 检测用户语音 持续接收麦克风输入	清空待播音频 打断当前 response*
AURA	按住说话，松开后 提交新的音视频输入	按住时停止本地播放 服务端仍可继续生成

PersonaPlex：显式 cancel 会重置对话上下文；断连后需新建 session 。

模型自主发言

PersonaPlex • MiniCPM-o 4.5
模型决定何时发言

本地停播

AURA 按钮 • Qwen VAD demo
停止本地播放，推理仍可能继续

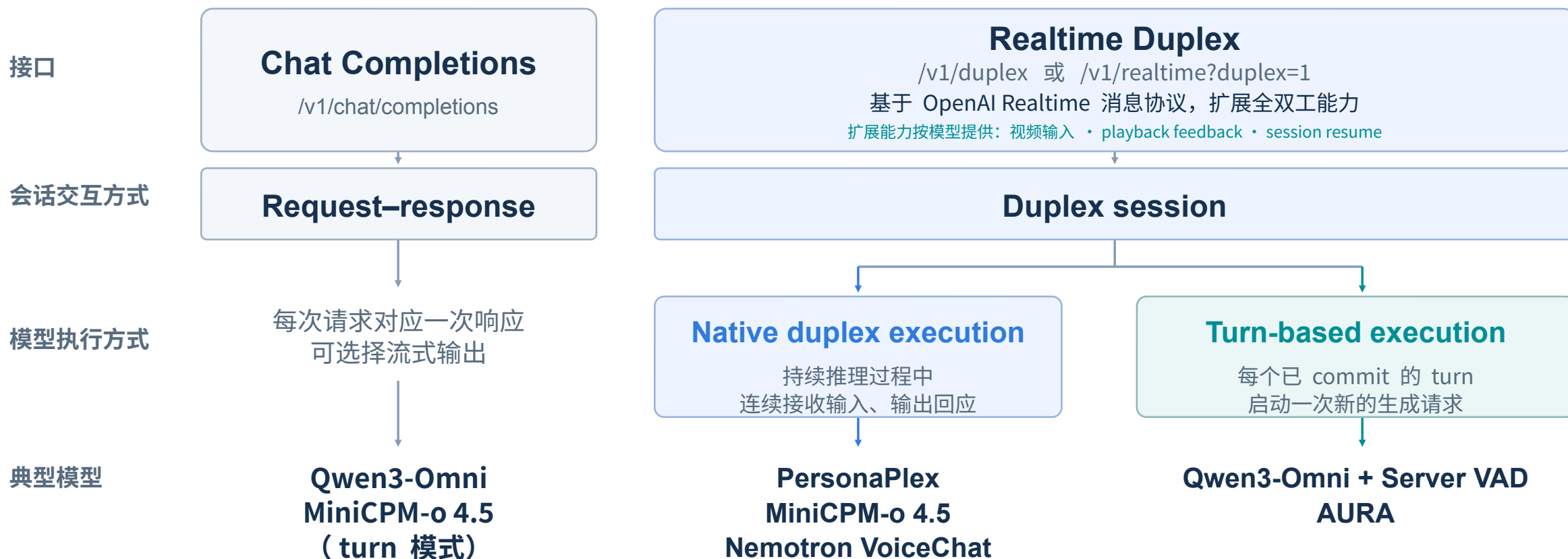
服务端取消

Qwen3-Omni + Server VAD*
取消当前生成

* 各模型的 demo 控制方式不同；Qwen VAD 取消需启用 interrupt_response 。

接口、会话交互与模型执行方式

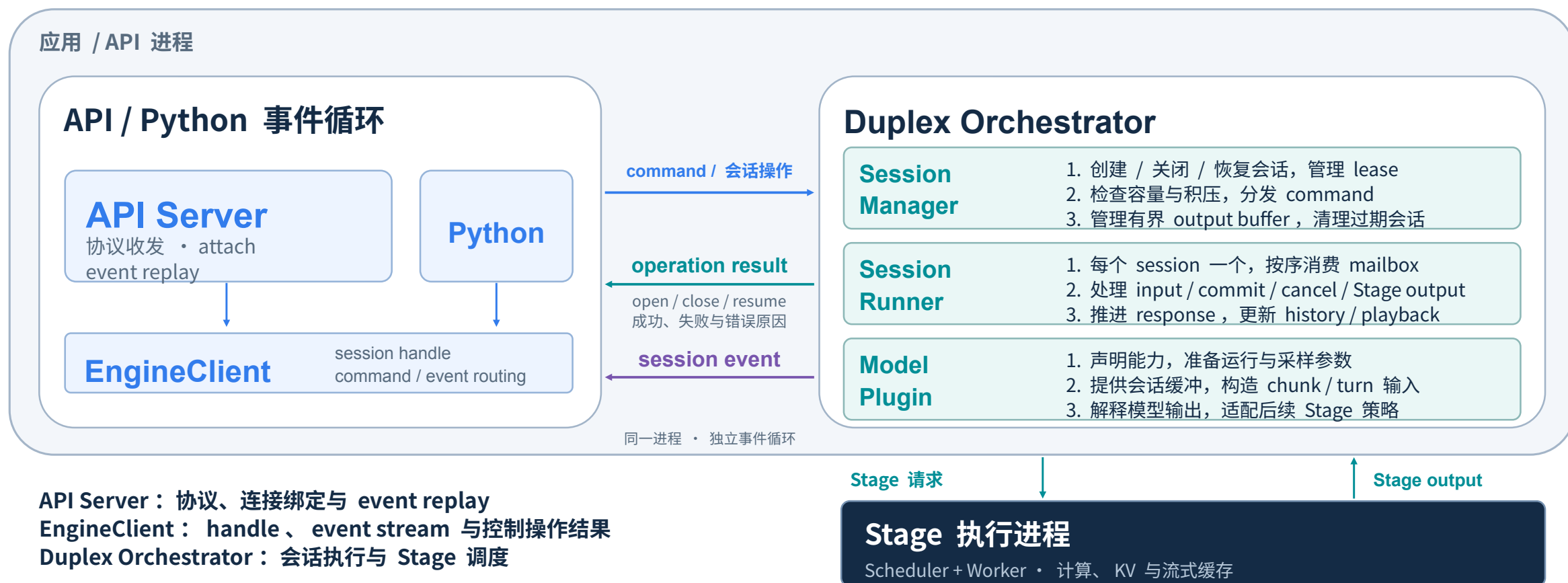
接口承载会话交互；执行方式决定模型如何处理输入。



Realtime 另有独立的 turn-based 路径：/v1/realtime?duplex=0。

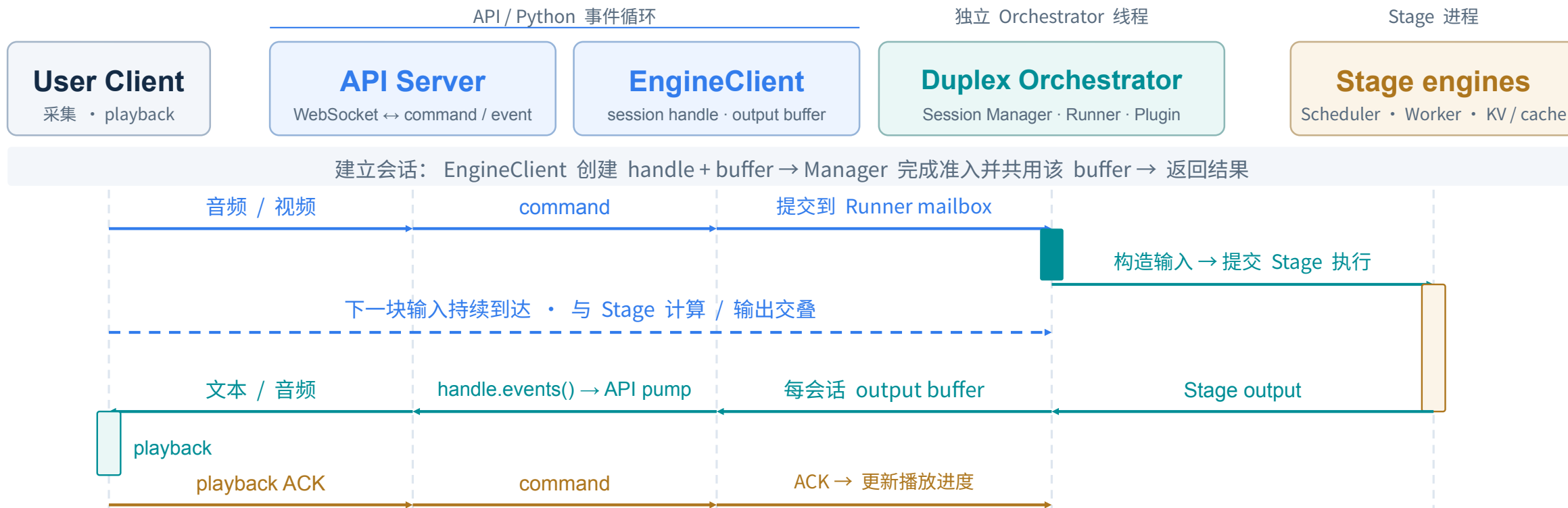
统一全双工架构，通信、会话控制与模型计算按层分工

Duplex WebSocket 与 Python 会话共用引擎侧会话控制、模型插件和有界输出传输。



全双工 workflow

API Server 处理协议；EngineClient 路由 command 与 event；Orchestrator 协调会话执行。



Native：追加到持续请求 · Turn-based：commit 后启动新请求 · 两种模式均可持续接收输入

Session event ACK 在 API 层处理；playback ACK 进入 Runner，history 更新依模型能力而定。

从实时输入到语音输出：Stage 与状态

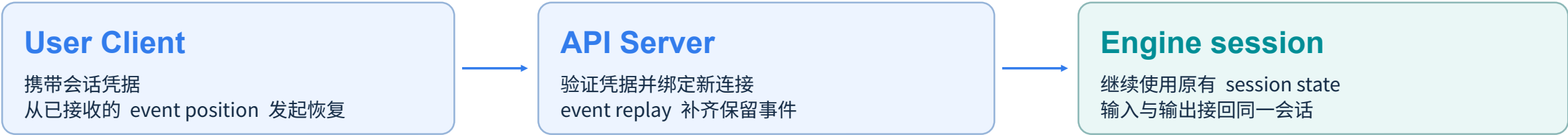
native duplex 续用计算状态； turn-based execution 根据历史构建新请求。



PCM = 音频波形 · Audio tokens = codec 编码 · KV = 模型 attention cache

连接恢复：按模型能力处理

支持 session resume：MiniCPM-o 4.5、AURA、Qwen3-Omni 与 Nemotron VoiceChat。



在 session 有效期间，重新连接到同一个仍存活的 API 进程。
事件 replay 依赖保留窗口；进程重启后需新建 session。

恢复环节	框架行为	User Client 接入
会话身份	沿用同一 session	保存 session ID 与最新的 resume token； Python WebSocket SDK 会管理这两项凭据。
连接绑定	新连接接管原会话	新连接的首条消息发送 session.resume； 保存 session.resumed 返回的新 token。
事件续传	续传保留的 event	提供最后接收的 event sequence。 若所需事件已缺失，则新建 session。

PersonaPlex demo：断连后新建 session，并重新应用原有音色与 persona。

性能优化方法与持续改进

减少重复工作，让 GPU 更多时间用于有效计算。

CUDA Graphs：减少重复调度

复用准备好的 GPU 执行流程，降低 CPU 调度开销。

MiniCPM-o 4.5 · Qwen3-Omni (图像) · PersonaPlex

Kernel fusion：合并小操作

减少 GPU 任务启动次数和中间数据搬运。

MiniCPM-o 4.5

Session batching：多路会话一起算

将多个会话的新音频帧合成一批处理。

PersonaPlex

Incremental decoding：只解码新音频

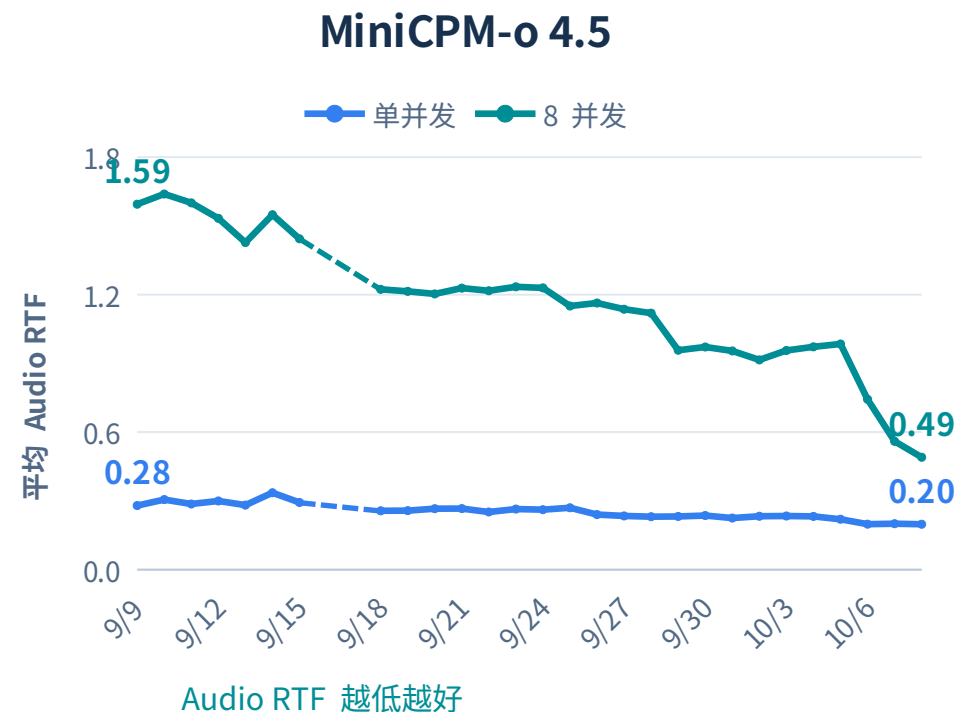
保留解码状态，只处理新增的音频 chunk。

PersonaPlex

Seed-TTS · 2026 年 9 月 9 日至 10 月 8 日；虚线跨越无同配置测试记录的日期。

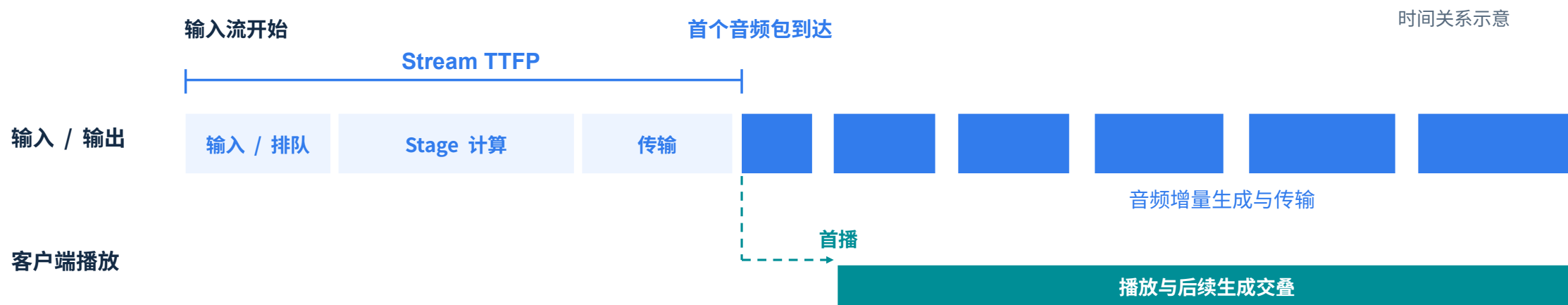
Audio RTF = 音频生成耗时 / 输出音频时长，按请求取平均。

H100 · Chat Completions



从首包速度到持续交互质量

结合 OmniInteract、Omni-DuplexEval 与延迟、流式输出指标，评估实时交互。



首音频延迟

Stream TTFP • p50 / p95
输入开始 → 收到首个音频包

持续输出

Response RTF • chunk 间隔
停顿与播放缓冲

长会话与并发

持续时长 • session 数
峰值显存 • 资源回收

交互质量

何时回应 • 回应内容
音视频理解

TTFP 衡量首包到达时间；真正何时开始回答，需结合交互测评判断。
比较指标时需统一计时窗口；测评覆盖范围因模型而异。

全双工模型的工具调用

基于已有工具调用路径，扩展更多模型在统一双工框架中的支持。

当前已支持

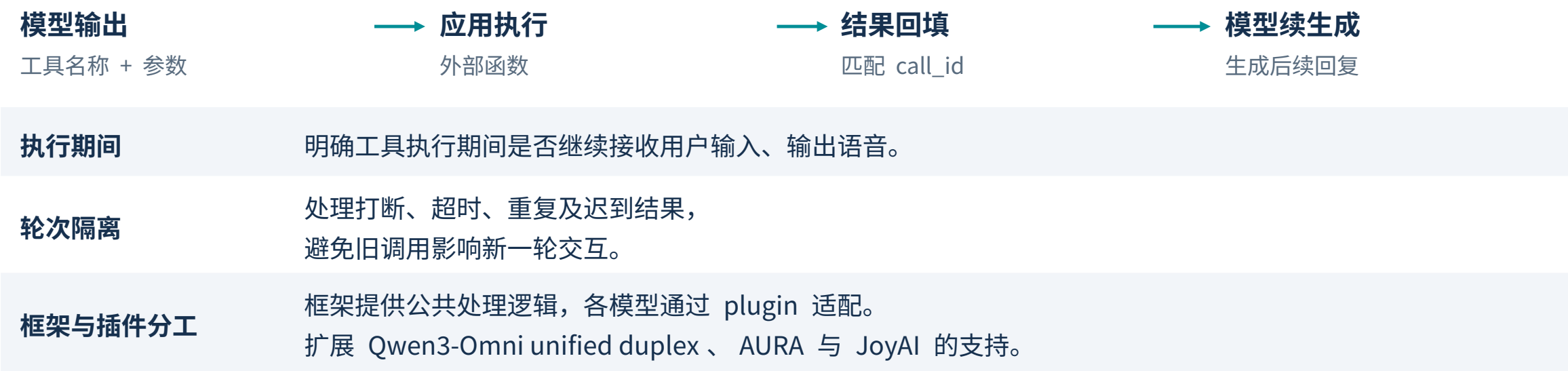
Qwen3-Omni Realtime

工具调用解析 • /v1/realtime?duplex=0

Nemotron VoiceChat

统一 native duplex 中的工具调用与结果回填

下一步：扩展统一双工工具调用支持



全双工 roadmap

近期完善模型接入、交互能力与测评；远期扩展会话时长和服务规模。

近期

JoyAI 接入

将 JoyAI 接入 unified duplex 框架
完成模型输入与输出适配

通用工具调用

扩展更多模型的工具调用支持
处理打断、超时与迟到结果

Semantic VAD

结合语义与上下文判断用户是否说完
接入 commit 与 response 策略

扩展实时交互测评

扩展 OmniInteract / Omni-DuplexEval 覆盖
增加模型、交互任务与回归检查

远期

更长且稳定的连续会话

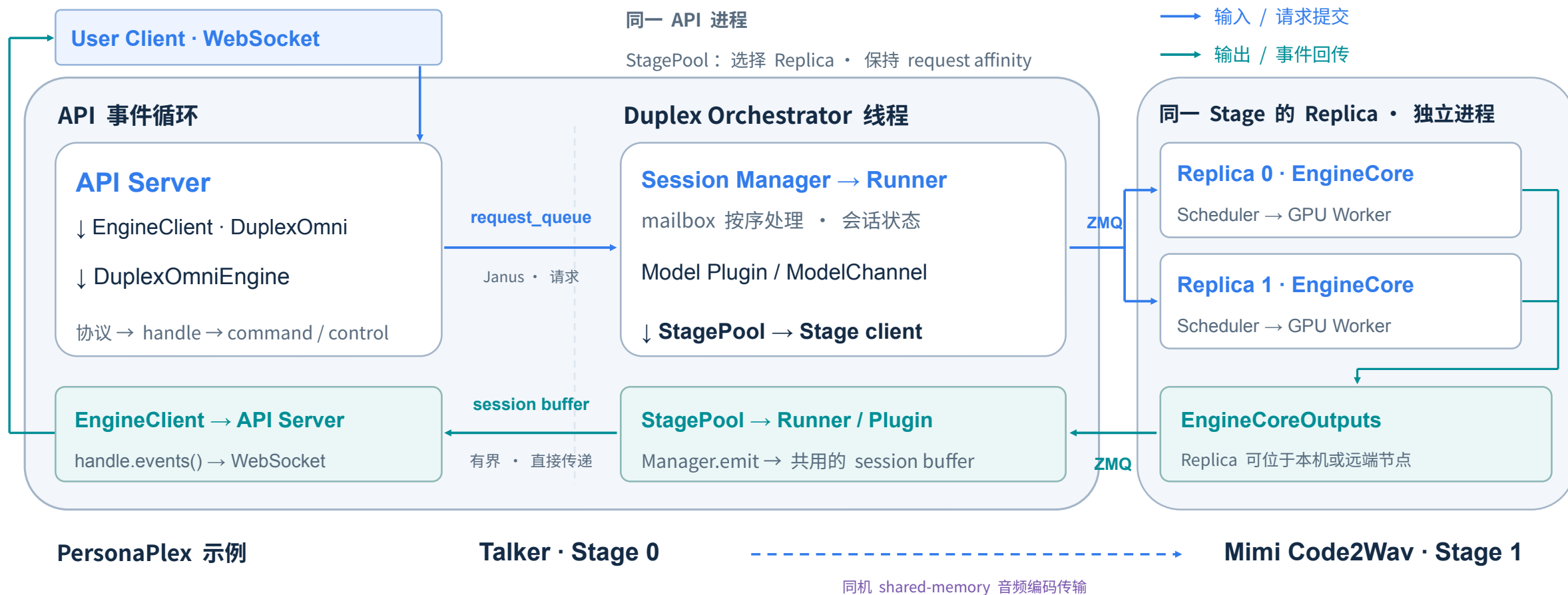
管理长时间音视频会话的 context 与 history
随会话延长，保持稳定的延迟与内存占用

扩展到多个 API worker

在现有 Stage replica 支持上继续扩展
增加跨 API frontend 的会话路由与归属管理

全双工执行链路：从输入到模型，再到输出

普通输出通过每个 session 的有界 buffer 传递，由 API 与 Orchestrator 线程共用。



模型发言决策 与 VAD 打断

01 模型决定何时发言

新输入持续进入模型计算。
模型自主决定继续发言或让出话语权。

02 VAD 驱动的 turn-taking

检测到用户发声时，打断当前回复。
静音达到阈值后 commit，启动新请求。

两种取消入口

显式 `response.cancel`

客户端 `command` 进入会话取消处理。
底层取消逻辑停止当前生成。

同一段实时会话

摄像头



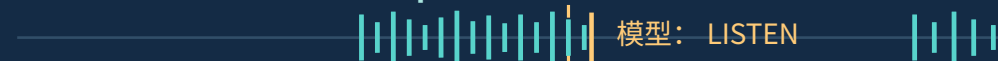
用户语音



Barge-in : 用户插话



MiniCPM-o 4.5 · native duplex



模型决定 LISTEN / SPEAK ; 已排队音频可继续播放。

Qwen3-Omni + VAD · turn-based



用户发声停止生成与播放；静音达到阈值后 commit。

LISTEN 是模型决策，与这两条路径不同。

VAD 触发打断

用户发声触发 overlap policy。
复用底层取消逻辑；客户端停止播放。

Acoustic VAD 与 Semantic VAD

常见组合：Acoustic VAD 检测停顿，再由 turn-end detector 判断用户是否说完。

对比维度	Acoustic VAD	Semantic VAD 规划扩展
判断依据	语音活动 + 配置的静音时长	语义与上下文 音频模型还可结合语调等线索
停顿处理	静音达到阈值时可能 commit 即使句子尚未说完	判断话语尚未完整时 可继续等待
系统动作	可在静音后自动 commit。 按配置决定是否生成 response。	判断用户说完后 commit 输入 按配置自动开始 response
计算与部署	CPU 推理开销低 当前：CPU thread pool	轻量模型可在 CPU 上运行 较重模型：计划接入 Omni Stage

开源组合示例：SILERO VAD + PIPECAT SMART TURN v3.2

“帮我订明天的…… [停顿] ……去北京的机票。”

1. Silero：区分语音 / 非语音，检测停顿

2. Smart Turn：判断话轮完整 / 尚未完整

Server VAD：Runner 执行 commit / cancel 策略；检测计算在其事件循环之外运行。

VAD 可减少非语音噪声造成的误触发。音频降噪与打断意图识别是独立能力。

断连、cancel 与 close ，各自影响什么

MiniCPM 、 AURA 、 Qwen3-Omni 与 Nemotron 支持活跃会话续接； PersonaPlex 断连即关闭会话。

触发操作	正在执行的工作	会话与恢复
断连 · 支持续接的模型	可继续生成；记录后续事件	断连 grace 内、会话仍存活时可续接
response.cancel	终止生成，丢弃尚未发出的输出	session 保留； PersonaPlex 重置模型上下文
session.close / lease 过期	终止并清理关联请求	会话结束；重新交互需新建

会话仍存活

在断连 grace 内恢复
session 仍由存活的 owner 持有

会话凭据有效

session ID + 最新 resume token
成功续接后 token 会更新

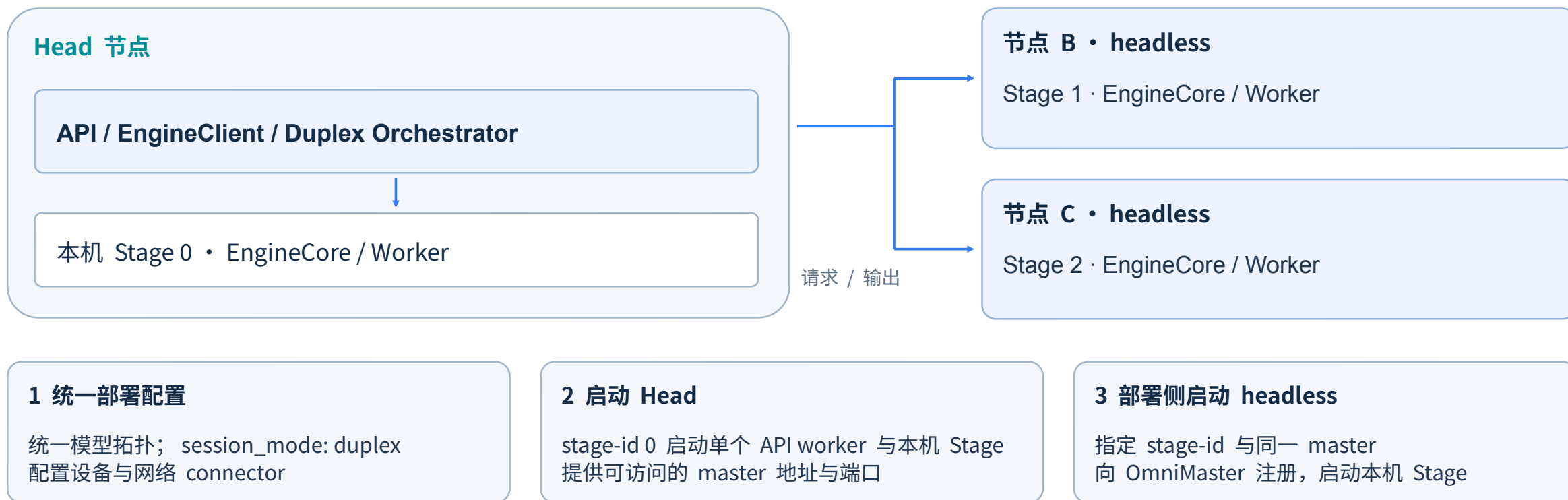
保留事件可续传

最后接收的 event sequence
所需事件缺失时需新建 session

默认：断连 grace 30s · idle TTL 300s · replay 60s / 8 MiB 。
客户端需另行停止已缓存的播放音频； session resume 依赖同一个仍存活的 API 进程。

跨机 Stage 共用入口侧的 Orchestrator

单个 API worker 持有 duplex session；Head 连接本机及已注册的 headless Stage 进程。



Headless Stage 共用 Head Orchestrator；duplex 不支持多 API worker，跨机传输需配置网络 connector。

参考代码

层次	实现入口	阅读重点
API 通信适配	OmniDuplexSessionHandler	协议 event、连接绑定与 replay
EngineClient	DuplexOmni / DuplexSessionHandle	handle、共享 output buffer 与 event
通信与传输	DuplexOmniEngine / DuplexOutputBuffer	控制队列与有界 session 输出
会话运行时	DuplexSessionManager / DuplexSessionRunner	lease、状态、mailbox 与模型策略
Stage 编排	DuplexOrchestrator / StagePool	初始请求、后续更新与副本绑定
模型执行	Scheduler / Model Worker	batching、计算与模型缓存